



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

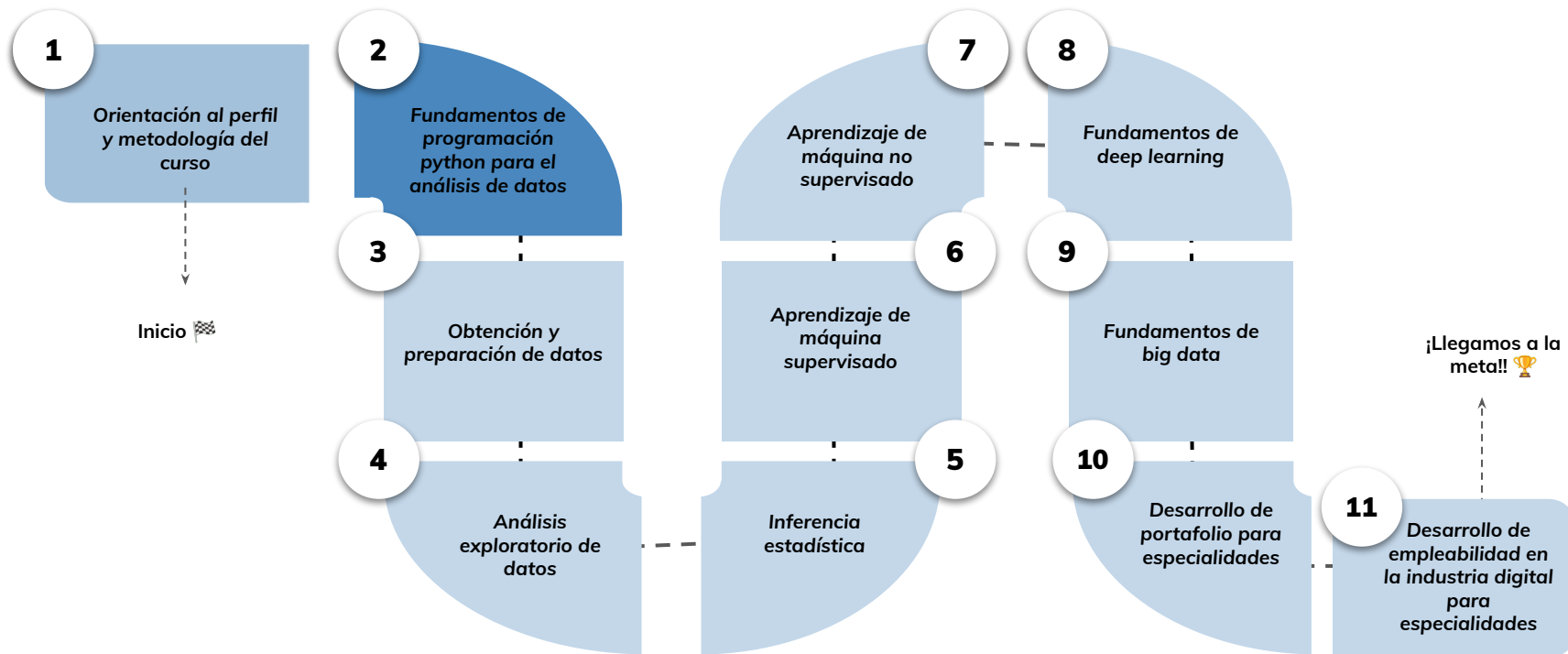


› El lenguaje Python- Parte 1

Aprendizaje Esperado 1: Explicar las principales herramientas del lenguaje python para resolver distintas problemáticas en el entorno de trabajo.

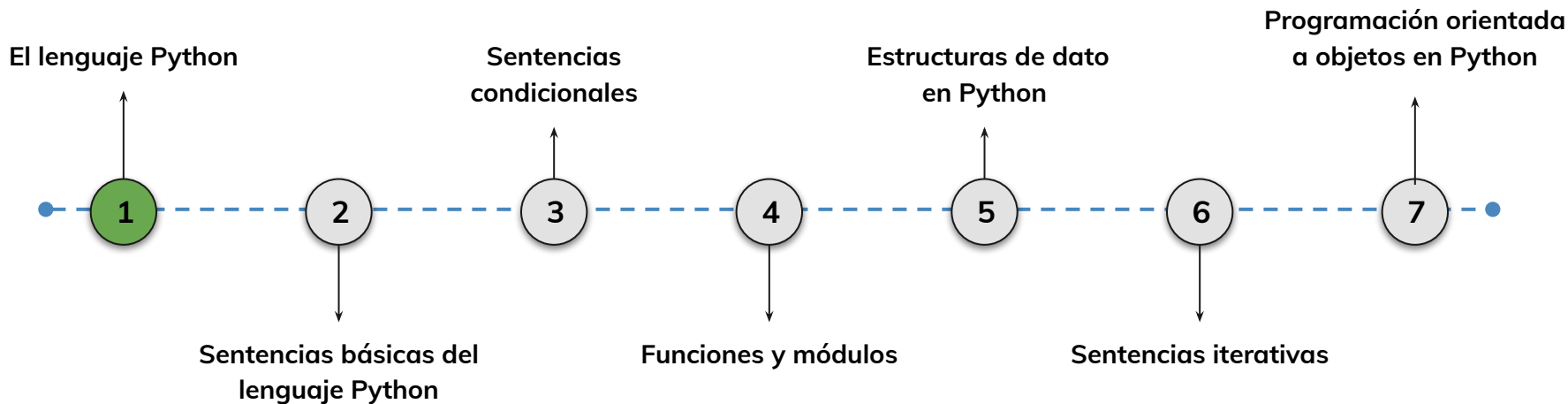
Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



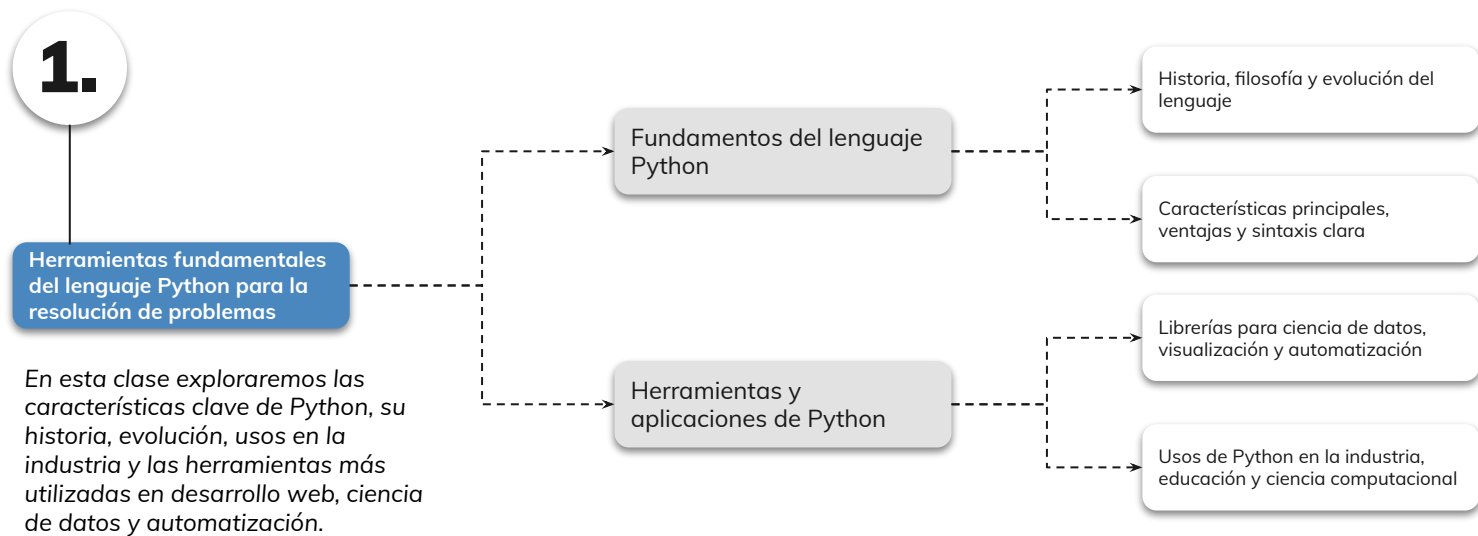
Roadmap de lecciones

¿Cuáles *lecciones* estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?





Objetivos de aprendizaje

¿Qué aprenderás?



- Conocer el propósito, historia y evolución del lenguaje Python.
- Comprender las principales características y ventajas de su sintaxis.
- Identificar las versiones relevantes y su impacto en la industria.
- Reconocer las herramientas y librerías clave de Python según el entorno de trabajo.
- Aplicar criterios para seleccionar librerías y frameworks de acuerdo al tipo de proyecto.

EL LENGUAJE PYTHON

Python es un lenguaje de programación de alto nivel, ampliamente utilizado en diversas áreas como desarrollo web, ciencia de datos, inteligencia artificial, automatización y desarrollo de software en general. Su facilidad de aprendizaje, sintaxis clara y versatilidad han contribuido a su popularidad tanto entre principiantes como entre programadores avanzados.



python

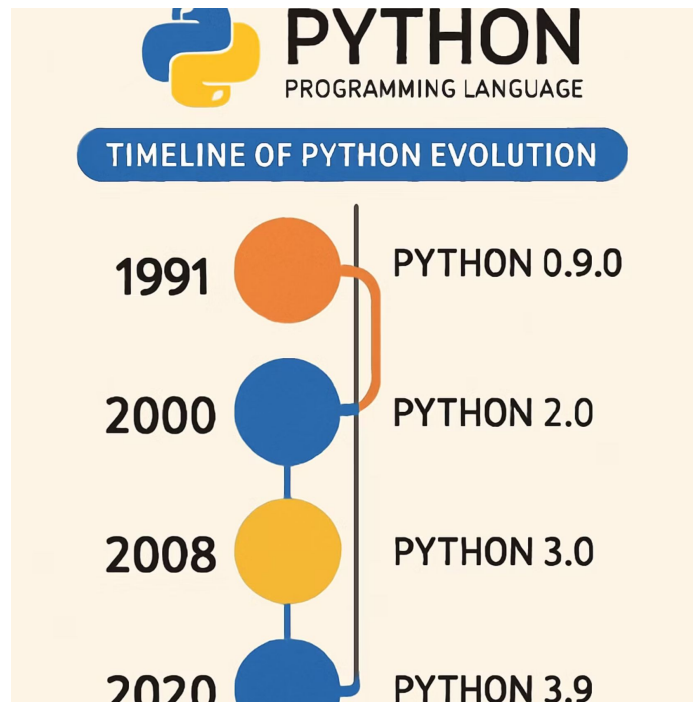
Introducción a Python

- Python permite a los desarrolladores escribir código de manera rápida y eficiente, sin comprometer el rendimiento, lo que lo convierte en una excelente elección para proyectos de cualquier escala.
- El lenguaje Python fue creado por Guido van Rossum en la década de 1980, y desde entonces ha evolucionado constantemente, gracias a su comunidad activa y al soporte de desarrolladores de todo el mundo.
- Hoy en día, Python es uno de los lenguajes de programación más utilizados, respaldado por una vasta cantidad de recursos, bibliotecas y frameworks que facilitan su aplicación en áreas emergentes como el machine learning y la ciencia de datos.

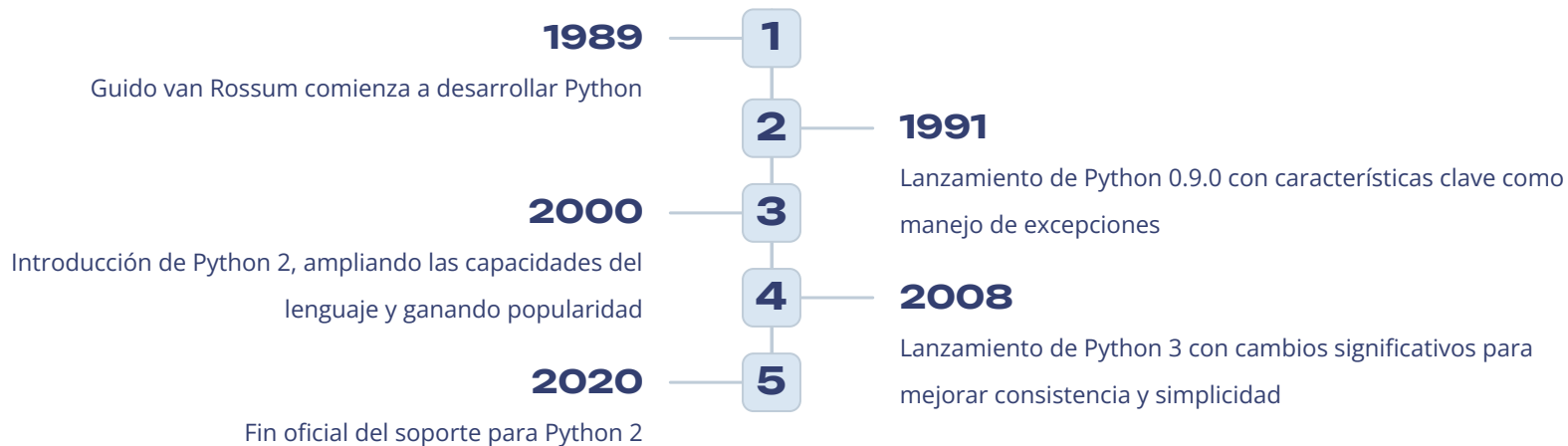
Historia y Creación del Lenguaje Python

Python fue creado por Guido van Rossum en 1989 como una alternativa a lenguajes de programación más complejos y menos accesibles. Inspirado por el lenguaje ABC y con la intención de ofrecer una sintaxis simple y legible, Van Rossum desarrolló Python como un lenguaje fácil de aprender pero suficientemente potente para resolver problemas complejos.

La primera versión oficial, Python 0.9.0, se lanzó en 1991 e incluyó características clave como el manejo de excepciones, funciones y los módulos.



Evolución de Python





Consolidación y Comunidad de Python

Uno de los factores clave en el éxito de Python es su activa y diversa comunidad, que incluye programadores, científicos, académicos y empresas. Esta comunidad contribuye con nuevas bibliotecas, herramientas y mejoras al lenguaje, garantizando que Python siga siendo relevante y responda a las demandas de la industria.

El soporte comunitario ha hecho posible la creación de bibliotecas ampliamente utilizadas como NumPy, Pandas, TensorFlow y Django, que extienden las capacidades de Python en áreas como ciencia de datos, machine learning y desarrollo web.

Estándares y Organización

La comunidad de Python también mantiene convenciones y estándares de programación, como el PEP 8, que establece directrices para escribir código claro y legible.

La organización sin fines de lucro Python Software Foundation (PSF) desempeña un papel crucial en la promoción y desarrollo del lenguaje, apoyando conferencias como PyCon y ofreciendo recursos educativos.



Popularidad y Aplicaciones de Python

La simplicidad y versatilidad de Python han impulsado su popularidad en una amplia gama de aplicaciones. Python se ha convertido en el lenguaje preferido para la ciencia de datos y la inteligencia artificial debido a su facilidad para realizar cálculos numéricos, procesar grandes volúmenes de datos y trabajar con modelos de machine learning.



Desarrollo Web

Frameworks como Django y Flask facilitan la creación de aplicaciones web robustas y escalables



Inteligencia Artificial

Bibliotecas como TensorFlow y PyTorch permiten desarrollar modelos avanzados de IA



Ciencia de Datos

Herramientas como Pandas y NumPy facilitan el análisis y procesamiento de datos

Python en la Industria

Empresas líderes como Google, Facebook, Instagram y Netflix utilizan Python en sus proyectos, lo que demuestra su capacidad para satisfacer demandas empresariales de alto rendimiento. Además, su versatilidad y su integración con otros lenguajes lo han convertido en un estándar en muchas industrias.



Python en la Educación y el Aprendizaje

Python es ampliamente reconocido como un lenguaje accesible para los principiantes debido a su sintaxis intuitiva y su semejanza con el lenguaje natural. Este aspecto hace que Python sea el lenguaje de elección en muchos programas educativos y cursos de programación, donde los estudiantes pueden aprender los fundamentos de la programación de manera rápida y eficiente.

Su enfoque en la claridad y legibilidad facilita el aprendizaje de conceptos complejos sin que los estudiantes se vean abrumados por la sintaxis.



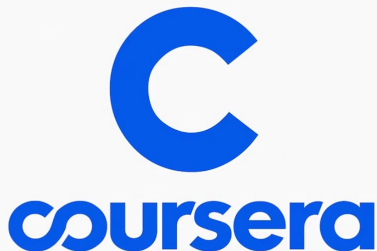
Recursos de Aprendizaje

Python cuenta con una vasta cantidad de recursos y documentación que respaldan su aprendizaje autodidacta.



Codecademy

Plataforma interactiva con cursos estructurados de Python



Coursera

Cursos universitarios y especializaciones en Python



GitHub

Repositorios de código para aprender y colaborar en proyectos reales

Limitaciones y Desafíos de Python

Rendimiento

Al ser un lenguaje interpretado y no compilado, puede ser más lento en comparación con otros lenguajes como C++ o Java. Esto puede ser un obstáculo en aplicaciones donde el tiempo de ejecución es crítico.

Gestión de memoria

Python es menos eficiente en la gestión de memoria, lo que puede ser un problema en aplicaciones de gran escala.

Soluciones

Estas limitaciones se pueden mitigar mediante el uso de extensiones y módulos específicos, como Cython y Numba, que permiten compilar partes del código en C para mejorar el rendimiento.

Python como Lenguaje Multipropósito

Python es un lenguaje de programación multipropósito, lo que significa que puede ser utilizado en una amplia variedad de aplicaciones, desde desarrollo web hasta ciencia de datos. Esta versatilidad permite que Python sea adoptado en múltiples sectores, incluidos los negocios, la ciencia, la ingeniería, y la educación.

El propósito de Python es ofrecer un lenguaje que sea fácil de leer y de escribir, pero que a su vez sea lo suficientemente potente para resolver problemas complejos en distintos dominios.



WEB



DATA SCIENCE



AUTOMATION

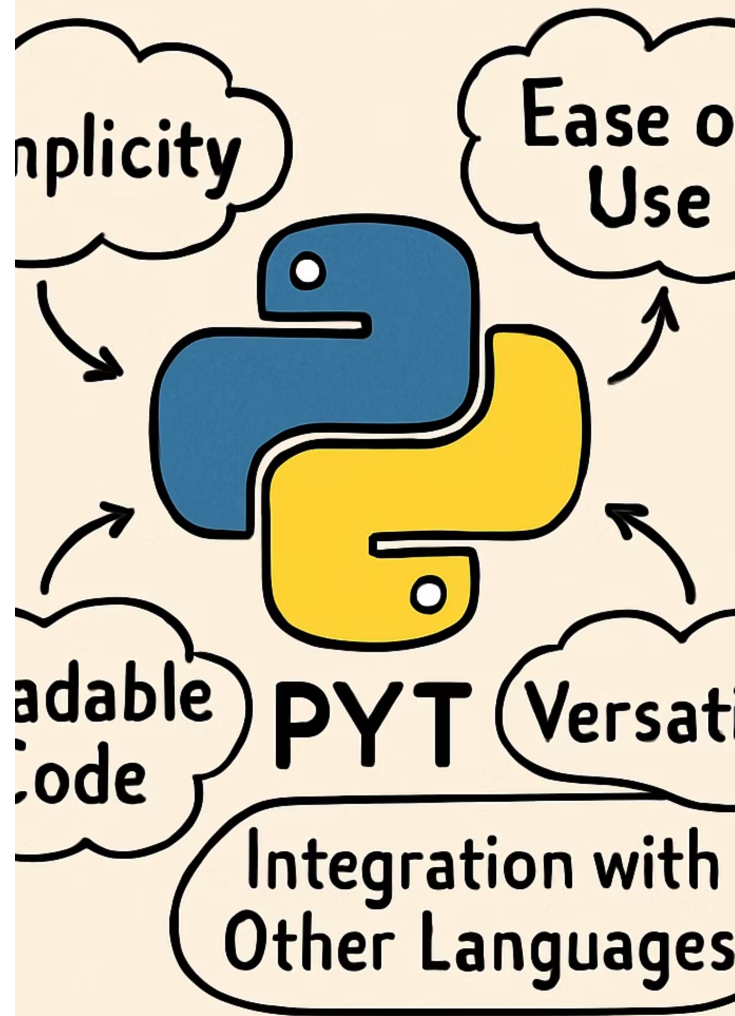


PROFESSIONAL
LIGHTING

Filosofía de Python

La filosofía de Python se enfoca en la simplicidad y en facilitar el desarrollo de programas robustos. Esto lo hace ideal para desarrolladores de todos los niveles, desde principiantes hasta expertos.

La posibilidad de integrar Python con otros lenguajes y tecnologías también contribuye a su amplio uso en proyectos de diferentes tipos y tamaños.



Frameworks Web de Python



Django

Conocido por su capacidad para simplificar el desarrollo de sitios web complejos con un enfoque "baterías incluidas"



Flask

Preferido en proyectos que requieren un enfoque más ligero y flexible, ideal para APIs y aplicaciones pequeñas



Pyramid

Ofrece flexibilidad y escalabilidad para proyectos que crecen con el tiempo

El desarrollo web en Python permite que los desarrolladores trabajen de manera eficiente con bases de datos, manejo de formularios y autenticación de usuarios, simplificando muchas de las tareas comunes en el desarrollo de aplicaciones.

Python en la Ciencia de Datos y el Machine Learning

Uno de los propósitos más destacados de Python en la actualidad es su aplicación en la ciencia de datos y el machine learning. Gracias a bibliotecas especializadas, Python permite a los científicos de datos y a los ingenieros de machine learning trabajar con grandes conjuntos de datos, realizar análisis complejos y construir modelos de aprendizaje automático con facilidad.



Bibliotecas para Ciencia de Datos



Pandas

Manipulación y análisis de datos estructurados con estructuras como DataFrames



NumPy

Computación numérica eficiente con arrays multidimensionales



Scikit-learn

Herramientas simples y eficientes para análisis predictivo y modelado estadístico



TensorFlow

Biblioteca de código abierto para aprendizaje automático y redes neuronales

Python en la Investigación

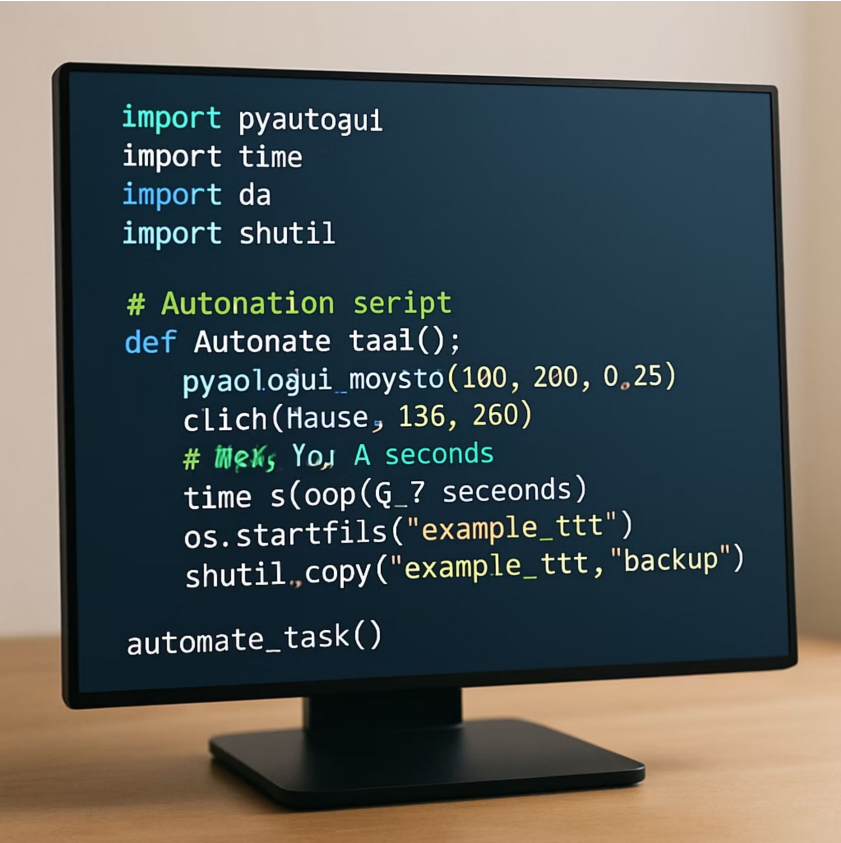
La capacidad de Python para integrarse con entornos de computación de alto rendimiento y herramientas como Jupyter Notebooks ha impulsado su adopción en el mundo de la investigación. Python permite a los profesionales de datos realizar experimentos rápidamente, visualizar resultados y comunicar hallazgos de manera clara, lo cual es esencial en el ámbito científico.



Python en la Automatización y el Scripting

Python es ideal para tareas de automatización y scripting, donde se requiere realizar tareas repetitivas o controlar otros sistemas. Con solo unas pocas líneas de código, los programadores pueden automatizar procesos como:

- Manipulación de archivos
- Acceso a servicios de red
- Extracción de datos de sitios web
- Administración de servidores

A photograph of a black computer monitor on a wooden desk. The monitor screen displays a Python script with syntax highlighting. The code includes imports for pyautogui, time, and shutil, followed by a function definition 'def Automate task()' which contains logic for clicking a mouse button, waiting for a specified time, and copying a file to a backup directory. The function is then called at the bottom of the script.

```
import pyautogui
import time
import os
import shutil

# Automation script
def Automate task():
    pyautogui.click(100, 200, 0.25)
    click(House, 136, 260)
    # Wait for A seconds
    time.sleep(5 seconds)
    os.startfile("example_ttt")
    shutil.copy("example_ttt", "backup")

Automate task()
```


Ventajas en Automatización

La facilidad para escribir scripts y la disponibilidad de librerías de automatización han convertido a Python en una de las opciones preferidas para el desarrollo de herramientas y procesos que simplifican tareas complejas.



Pruebas de sistemas

Python se utiliza para crear scripts de prueba automatizados que verifican el funcionamiento correcto de aplicaciones



Monitoreo

Scripts de Python pueden supervisar sistemas y enviar alertas cuando se detectan problemas



Integración continua

Python facilita la automatización de procesos de integración y despliegue continuo en el desarrollo de software

Python Programming

```
# define a function
def greet(name):
    print(f"hello, {name}")

# Call the function
greet("Alice")

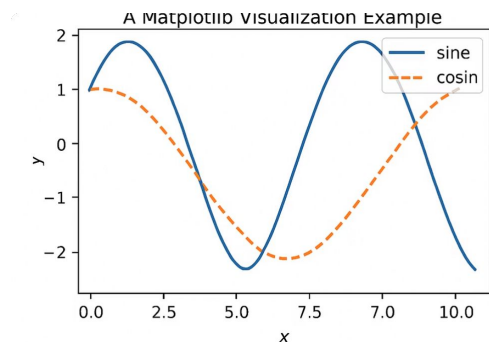
Hello, Alice!
```

Python en la Educación y la Ciencia Computacional

Python es también el lenguaje preferido en la educación y en proyectos científicos, ya que proporciona una plataforma sencilla para enseñar conceptos de programación. En la ciencia computacional, Python facilita el desarrollo de simulaciones y la creación de modelos complejos, lo que lo convierte en una herramienta valiosa para investigadores y educadores.

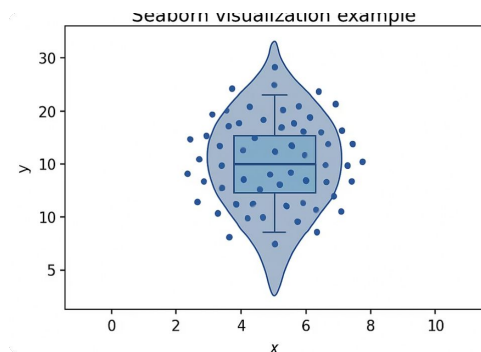
Visualización de Datos en Python

Python ofrece una extensa biblioteca de visualización de datos, que facilita la comunicación visual de resultados científicos.



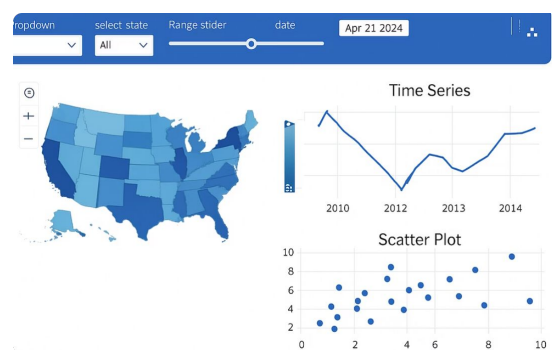
Matplotlib

Biblioteca de visualización estándar para gráficos científicos



Seaborn

Interfaz de alto nivel para crear visualizaciones estadísticas atractivas

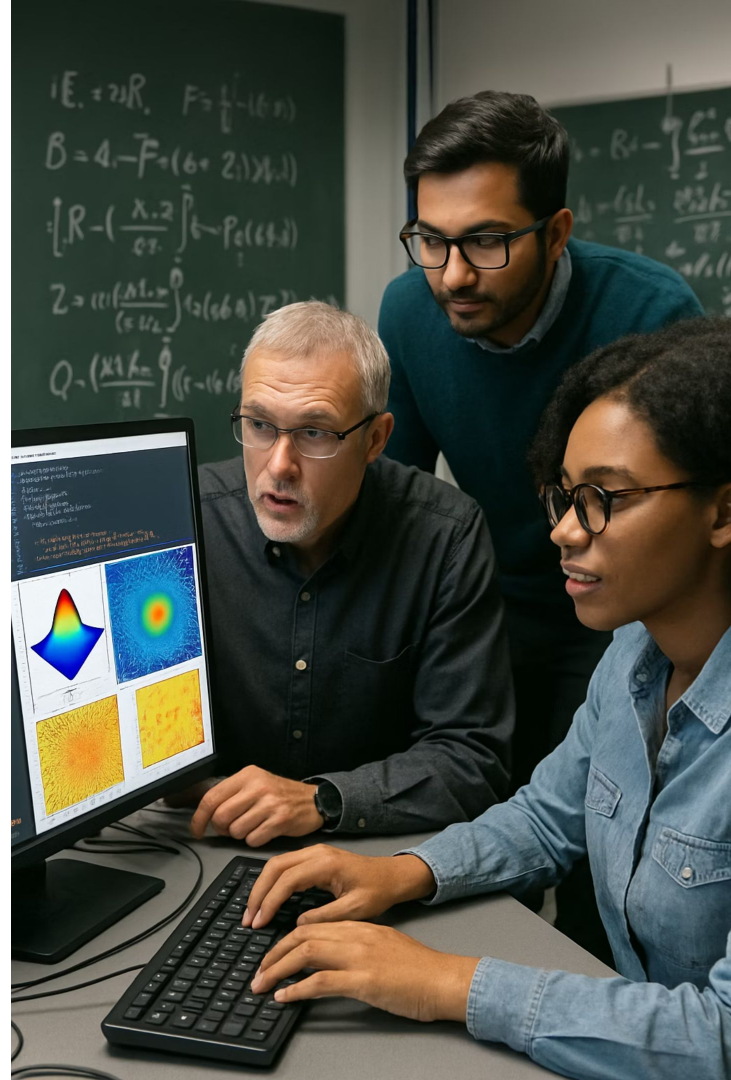


Plotly

Gráficos interactivos para aplicaciones web y dashboards

Python en Diversas Ciencias

La versatilidad de Python le permite abordar desde cálculos matemáticos hasta simulaciones complejas en física, biología y química. Su simplicidad y robustez han hecho que sea adoptado en cursos universitarios y programas de formación en ciencia de datos, inteligencia artificial y programación en general.



Sintaxis Clara y Sencilla

Una de las principales características de Python es su sintaxis clara y sencilla, que permite escribir código fácil de leer y de mantener. Python se enfoca en la legibilidad, utilizando indentación para definir bloques de código en lugar de llaves, lo que facilita la comprensión y minimiza errores comunes.

```
# Ejemplo de sintaxis clara en Python
def saludar(nombre): """Función simple que saluda a
una persona"""
    mensaje = f"Hola, {nombre}!"
    return mensaje
# Llamada a la función
print(saludar("Mundo"))
```

Ventajas de la Sintaxis de Python



Colaboración efectiva

La simplicidad permite que los equipos de desarrollo colaboren de manera efectiva, ya que el código Python suele ser fácil de entender incluso para quienes no participaron en su creación.



Reducción de errores

La claridad y consistencia en la sintaxis ayudan a reducir la cantidad de errores y facilitan el proceso de depuración.



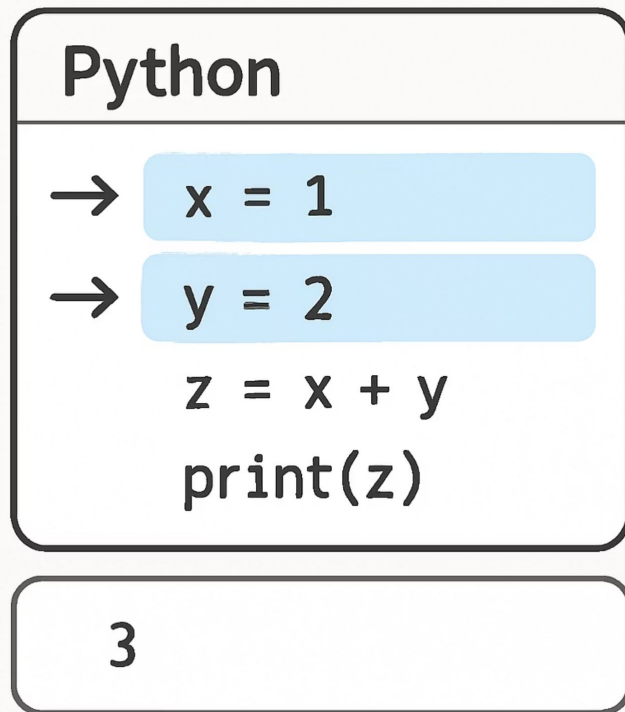
Curva de aprendizaje suave

La sintaxis intuitiva hace que Python sea accesible para principiantes, permitiéndoles concentrarse en los conceptos de programación en lugar de en las peculiaridades del lenguaje.

Interpretado y Dinámico

Python es un lenguaje interpretado, lo que significa que el código se ejecuta línea por línea sin necesidad de compilación. Esto permite una mayor flexibilidad en el desarrollo y facilita la realización de pruebas y depuración en tiempo real.

Al ser un lenguaje dinámico, Python permite definir y modificar tipos de datos de manera flexible, adaptándose al flujo de trabajo y permitiendo realizar cambios rápidamente.



Ventajas de ser Interpretado

El enfoque interpretado también facilita la ejecución en múltiples plataformas, ya que el intérprete de Python se encarga de manejar las diferencias entre sistemas operativos. Esto es particularmente útil en entornos de desarrollo donde se trabaja en distintas plataformas, como Windows, macOS y Linux.



Soporte para Múltiples Paradigmas

Python es un lenguaje multiparadigma, lo que significa que admite tanto programación orientada a objetos (POO) como programación funcional.



Programación Orientada a Objetos

Los programadores pueden organizar el código en clases y objetos, lo que permite estructurar el software de manera modular y reutilizable.



Programación Funcional

Python permite escribir funciones puras y aplicar conceptos de programación funcional, como el uso de funciones lambda, map(), filter(), y reduce().



Combinación de Paradigmas

Esta flexibilidad permite que los desarrolladores elijan el enfoque más adecuado para cada proyecto.

PYTHON STANDARD LIBRARY

Text Processing

re	difffib
diff	textwrap
unkodedata	uricodedata
string	

Data Types

daterime	heapo
amay	enum
enum	types
types	collections

Numeric and Mathematical

numbers	decimal
fractions	statistics
math	random

Functional Programming

iterioois	operator
functools	

File and Directory Access

pathlib	shutil
ds path	fimiatch
frimatch	glub

Networking and Interprocess

socket	asncio
--------	--------

Data Persistence

sqlites	dbm
dbm	shelve
marshai	pinckie

Software Development

traceback	2163
pob	doctest
unitrest	

Extensa Biblioteca Estándar

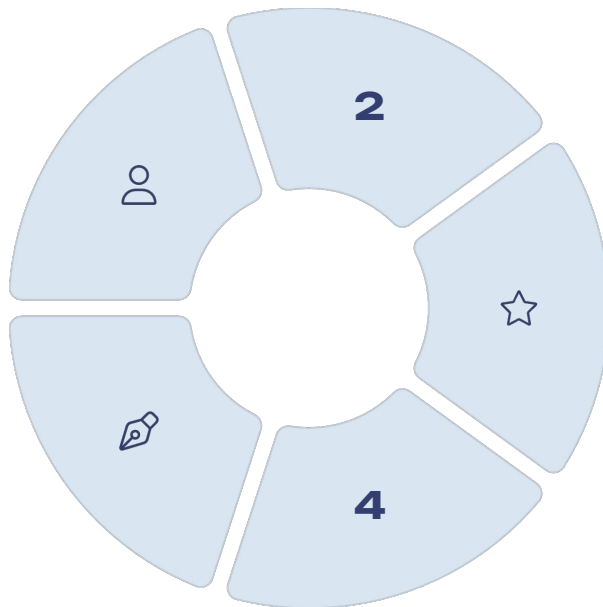
Python cuenta con una biblioteca estándar extensa que incluye módulos para manejo de archivos, procesamiento de texto, manipulación de fechas, operaciones matemáticas, y mucho más. Esta biblioteca permite realizar tareas comunes sin necesidad de instalar librerías adicionales, lo que agiliza el desarrollo.

Ecosistema de Librerías

Además de la biblioteca estándar, Python cuenta con un ecosistema de librerías externas que extienden las capacidades del lenguaje para aplicaciones especializadas.

NumPy
Computación científica y arrays
multidimensionales

TensorFlow
Aprendizaje automático y redes
neuronales



Pandas
Análisis y manipulación de datos
estructurados

Matplotlib
Visualización de datos y gráficos

Django
Framework para desarrollo web



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Explicación y resolución de Problemáticas

con Herramientas Python

Explicación y Resolución de Problemáticas con Herramientas Python

Contexto: 🙌

Has sido contratado/a por una startup tecnológica que trabaja con distintos proyectos de análisis de datos, desarrollo web y automatización de tareas. El equipo necesita orientación para seleccionar las herramientas adecuadas de Python según el tipo de problema.

Consigna: 📝

Selecciona 3 problemáticas comunes del entorno laboral (por ejemplo: análisis de datos, automatización de tareas repetitivas, o desarrollo de un portal web). Para cada una, responde:

- ¿Qué herramienta o librería de Python utilizarías?
- ¿Por qué es adecuada esa herramienta para el problema?

Tiempo 🕒: 30 minutos

Paso a paso: ⚙️

1. Piensa en los desafíos frecuentes en el ámbito profesional.
2. Relaciónalos con las herramientas vistas en clase (Pandas, Flask, NumPy, etc.).
3. Justifica tu elección con fundamentos técnicos.
4. Redacta una respuesta clara, orientada a alguien que está empezando en Python.

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Reconocimos la historia y evolución del lenguaje Python.
- ✓ Identificamos sus características clave como lenguaje multipropósito.
- ✓ Conocimos su sintaxis clara y su facilidad de aprendizaje.
- ✓ Exploramos aplicaciones concretas en ciencia de datos, web y automatización.
- ✓ Aprendimos a relacionar problemáticas reales con librerías y frameworks específicos.



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son claves para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

